

Play along on your phone

Scan, then type

```
python yatzy.py
```

neonemesis.github.io/intro

Yatzy in Python

A terminal game for 1–6 players

Victor Vilches & Axel Bohlmark

Presentation, October 2026

github.com/NeoNemesis/intro

What the game does: one turn

1–6	15	3	11
players	categories	rolls per turn	automated tests

1. Roll all **five dice**
2. Choose which dice to **hold**
3. Reroll the rest, **three rolls** max
4. Score in a **free category**

The game ends when everyone has filled their scorecard. Highest total wins. At least **63 points** on ones–sixes gives a **50-point bonus**.

Four files: logic and view kept apart

`yatzy.py`
Starts the game, catches Ctrl+C

↓

`terminal_view.py`
All input and output

↓

`game.py`
Game rules: Dice, Player, Game

↓

`scoring.py`
The categories and the scoring

`test_yatzy.py` tests `game.py` and `scoring.py` directly with **11 tests**

The rule we followed

`game.py` and `scoring.py` **never** use `print()` or `input()`.

The rules can be tested

The tests call the game logic directly, without anyone typing answers.

The view can be replaced

A graphical window could use the same rules. Only the view changes.



Dice: five dice that can be held

```
class Dice:
    def __init__(self, rng=None):
1       self.rng = rng if rng is not None else random.Random()
2       self.values = [1] * NUM_DICE
        self.held = [False] * NUM_DICE

    def roll(self):
        for i in range(NUM_DICE):
3           if not self.held[i]:
                self.values[i] = self.rng.randint(1, 6)

    def hold_only(self, positions):
4       self.held = [i in positions for i in range(NUM_DICE)]
```

1. The random generator can be **passed in**. The tests give it a fixed seed.
2. Two lists: the dice **values** and whether each die is **held**.
3. Only dice that are **not** held are rolled again.
4. A list comprehension: choosing dice 1 and 3 gives `[True, False, True, False, False]`.

Game: the rules are enforced in code

```
def roll(self):
    if not self.can_roll():                                  # 1
        raise RuntimeError("Inga slag kvar")
    if not self.has_rolled():                                # 2
        self.dice.release_all()
    self.dice.roll()
    self.rolls_left -= 1

def choose(self, category):
    if not self.has_rolled():                                # 3
        raise RuntimeError("Du måste slå minst en gång först")
    if category not in self.available_categories():
        raise ValueError("Kategorin är redan använd")
    points = self.preview(category)                          # 4
    self.current_player().scores[category] = points
    self.next_turn()
    return points
```

1. Rolling is only allowed with **rolls left**.
2. First roll of a turn: held dice are **released**.
3. You must have **rolled**, and the category must be **free**.
4. Points are saved and the turn passes on.

Why `raise`?

Illegal moves are stopped by the logic itself.

Scorecard, turn order and winners

```
class Player:
    def upper_sum(self):
1       return sum(self.scores.get(key, 0) for key in UPPER)

    def bonus(self):
2       return BONUS_POINTS if self.upper_sum() >= BONUS_LIMIT else 0

class Game:
    def next_turn(self):
3       self.current = (self.current + 1) % len(self.players)
        self.rolls_left = MAX_ROLLS
        self.dice.release_all()

    def winners(self):
4       best = max(player.total() for player in self.players)
        return [player for player in self.players
                if player.total() == best]
```

1. `scores` is a dict of category → points. `.get(key, 0)` gives 0 for empty boxes.
2. At least **63** gives a **50** bonus (three of each face).
3. `%` makes the turn wrap around: 0 → 1 → 2 → 0.
4. A **list** of winners: on a tie, all top scorers win.

Scoring: Counter counts the dice



5
3 times

3
2 times

`Counter([3, 3, 5, 5, 5])` → `{5: 3, 3: 2}`

```
    def score(category, values):
1       counts = Counter(values)

        if category in UPPER:
2           face = UPPER.index(category) + 1
            return face * counts[face]
        ...
```

1. All 15 categories are scored in **one** function. First it counts how many there are of each value.
2. `"treor"` (threes) is at index 2 in UPPER, plus 1 gives **3**.

Example: threes

`[3, 3, 1, 3, 6]` → 3 × 3 = **9**

The combinations: each rule asks about counts

Category	The rule in the code	Example → points
One pair	highest value that appears at least 2 times, × 2	`[2,2,5,5,1]` → **10**
Two pairs	exactly two different values that appear at least 2 times	`[2,2,5,5,1]` → **14** `[4,4,4,4,1]` → **0**
Three / Four of a kind	the value that appears at least 3 / 4 times	`[4,4,4,2,1]` → **12**
Small / Large straight	sorted dice are `[1,2,3,4,5]` / `[2,3,4,5,6]`	**15** / **20**
Full house	the counts, sorted, are exactly `[2, 3]`	`[3,3,5,5,5]` → **21** `[5,5,5,5,5]` → **0**
Chance	the sum of all dice	`[1,2,3,4,6]` → **16**
Yatzy	only one single value: `len(counts) == 1`	`[2,2,2,2,2]` → **50**

Every example in the table is also a test in `test_yatzy.py`.

The terminal view: what the player sees

```
+-------+  +-------+  +-------+  +-------+  +-------+
| o   o |  | o   o |  | o     |  | o   o |  |       |
| o   o |  | o   o |  |   o   |  | o   o |  |   o   |
| o   o |  | o   o |  |     o |  | o   o |  |       |
+-------+  +-------+  +-------+  +-------+  +-------+
    1          2          3          4          5
 SPARAD     SPARAD                SPARAD

Slag kvar: 1
Skriv numren på tärningarna du vill spara (t.ex. 135)
Enter = slå om alla, p = sätt poäng, v = visa protokollet
> 124
```

The game speaks Swedish: **SPARAD** = held · **Slag kvar** = rolls left · **Enter** = reroll all · **p** = score · **v** = view the scorecard

Dice as text

`PIPS` holds the dots for each face. `dice_lines` draws the row.

Flexible input

`parse_positions` reads `135`, `1 3 5` and `1,3,5` the same way.

Does not crash

Invalid input (`6`, `abc`) returns `None`, and the question is asked again.

The tests: 11 of 11 pass

11/11

tests OK with unittest

- **6** for the scoring rules
- **5** for the game flow
- We tested the view by **playing**

```
    def test_full_game(self):
        game = Game(["A", "B", "C", "D", "E", "F"],
                    random.Random(7))
        while not game.is_over():
            game.roll()
            game.choose(game.available_categories()[0])
        for player in game.players:
            self.assertEqual(len(player.scores),
                             len(CATEGORIES))
        self.assertTrue(len(game.winners()) >= 1)
```

1. **Fixed seed:** same dice on every run.
2. Plays a **full game** with 6 players.
3. Everyone has filled all **15 categories**.

What we learned and what we would improve

Lessons

- Keep **logic and view apart**: the code becomes easy to test and extend
- The right tool (`Counter`) makes rules **short and readable**
- A **fixed seed** makes randomness predictable in tests

Next steps

- `score()` is a long chain of `if`s. It could become a **dict** with one function per category
- A **graphical view**, e.g. with tkinter, that uses the same `Game`
- **Tests for the view**, e.g. for `parse_positions`

Thank you! Questions?



Play: python yatzy.py

Backup slides

Answers to questions we might get

Questions about the rules

Question	Answer
Why do five fives score 0 as a full house?	A full house needs two **different** values: the counts must be exactly 2 and 3. Five of a kind has the count 5.
Does a full house count as two pairs?	Yes. `[3,3,3,5,5]` has two values that appear at least twice → 3+3+5+5 = **16**.
Does four of a kind count as two pairs?	No. `[4,4,4,4,1]` has only **one** such value → 0 points. There is a test for it.
Can you score 0 points?	Yes, you can pick any free category, even if it gives 0 ("scratching" it).
What is the highest possible score?	**374**: upper section 105 + bonus 50 + one pair 12, two pairs 22, three of a kind 18, four of a kind 24, straights 15 + 20, full house 28, chance 30, Yatzy 50.
What happens on a tie?	`winners()` returns everyone with the top score, and the view prints a tie message ("Oavgjort mellan ...").



Questions about the code

Question	Answer
Why is `rng` passed into `Game`?	So the tests can give it a fixed seed, e.g. `random.Random(7)`. Otherwise a normal random generator is used.
Why `raise` when the view already checks?	The rules must hold whatever the view. A new view that forgets a check still cannot make an illegal move.
Why `triples[0]` and not `max()`?	With five dice only **one** value can appear three times or more. The same goes for four of a kind.
Why `release_all()` in both `roll` and `next_turn`?	`next_turn` releases the dice when the turn changes. The check in `roll` is an extra safety net.
How does the game know which categories are free?	`available_categories()` keeps those of the 15 that are missing from the player's `scores`.
What happens on Ctrl+C?	`yatzy.py` catches `KeyboardInterrupt` and prints "Spelet avbröts." ("The game was aborted.")



Python concepts used in the code

Concept	Where and what
`Counter`	`scoring.py`: counts how many times each value appears, e.g. `{5: 3, 3: 2}`.
List comprehension	`hold_only`: `[i in positions for i in range(NUM_DICE)]` builds a list in one expression.
`dict.get(key, 0)`	`upper_sum`: gets the value, or 0 if the key is missing.
`x if condition else y`	`bonus`: a condition in a single expression.
`%` (modulo)	`next_turn`: the remainder of a division makes the turn wrap back to player 0.
`raise`	`Game`: stops an illegal move with an error message.
f-strings	The view: `f"{player.name}s tur"` inserts values directly into the text ("Victor's turn").
`if __name__ == "__main__"`	`yatzy.py`: the code only runs when the file is started, not when it is imported.

Running the game and the tests

Start the game

```
cd yatzy
python yatzy.py
```

The files import each other directly (`from game import ...`), so they must be in the same folder.

Run the tests

```
cd yatzy
python -m unittest -v
```

Must be run inside the `yatzy` folder. From the repo root no tests are found, because `yatzy` is not a package.

Result: `Ran 11 tests ... OK`